Cloud Computing Course

Course n°3: Application on the cloud

Maximilien Walbecq
Data Engineer
Roquette Frères 🌱

Quiz !

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

1. What is a Blob Storage?

A. An object storage service for storing unstructured data like images & videos
B. A disk volume for VM
C. A messaging store for reliable messaging between application components.
D. A cache system for temporary data storage

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

1. What is a Blob Storage?

A. An object storage service for storing unstructured data like images & videos
B. A disk volume for VM
C. A messaging store for reliable messaging between application components.
D. A cache system for temporary data storage

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

2. What is a Virtual Network (VNET)?

A. A resource that segment networks for organization and security
B. A resource that control and filter inbound/outbound traffic
C. A resource that allows secure communication between resources
D. A private storage container for network files

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

2. What is a Virtual Network (VNET)?

A. A resource that segment networks for organization and security
B. A resource that control and filter inbound/outbound traffic
C. A resource that allows secure communication between resources
D. A private storage container for network files

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

3. What is IaC?

A. **I**nfrastructure **a**nd **C**loud: Cloud setup management
B. **I**dentity **a**nd **C**ontrol: Security and access manegement
C. **I**nfrastructure **a**s **C**ode: Manage and provide infrastructure using code
D. **I** H**a**te this **C**ourse: I don't want to come back

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

3. What is IaC?

A. **I**nfrastructure **a**nd **C**loud: Cloud setup management
B. **I**dentity **a**nd **C**ontrol: Security and access manegement
C. **I**nfrastructure **a**s **C**ode: Manage and provide infrastructure using code
D. **I** H**a**te this **C**ourse: I don't want to come back

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

4. What are the three main compute resources on the cloud?

A. Virtual Machines, Containers, and Serverless Functions
B. Disks, Databases, and Networks
C. Subscription, Resources Groups, and Load Balancers
D. Storage Accounts, Queues, and Pipelines

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

4. What are the three main compute resources on the cloud?

A. Virtual Machines, Containers, and Serverless Functions
B. Disks, Databases, and Networks
C. Subscription, Resources Groups, and Load Balancers
D. Storage Accounts, Queues, and Pipelines

© Maximilien WALBECQ - Created with Marp e MarpX.

Reminder of previous courses

Introduction to Cloud Computing

Aim of this session: Discover the Cloud, its features, services, and deployment models.

- Advantages of Cloud vs On-premise
- Data Center
- Virtualization
- Characteristics: Self-Service / Network Access / Elasticity / Measured / Resource Pooling
- Service: IaaS / PaaS / SaaS
- Deployment: Public / Private / Community / Hybrid

© Maximilien WALBECQ - Created with Marp e MarpX.

Reminder of previous courses

Infrastructure as a Service & as Code

Aim of this session:

- Understand IaaS fundamentals.
- Learn the basics of Infrastructure as Code and why it's important in cloud infrastructure management.
- Try to deploy an IT architecture through various tools

© Maximilien WALBECQ - Created with Marp e MarpX.

Now that we have explored *infrastructure management*, how can we create, deploy, and scale *applications* on these?

Today's session

Aim of this session:

- Explore the compute resources from the last session in more detail
- Discover the PaaS solutions
- Try to deploy applications on the cloud

© Maximilien WALBECQ - Created with Marp e MarpX.

Contenu

Platform as a Service

1. Title
2. Reminder of previous courses
3. Application Deployment on IaaS
4. Container-Based Deployment
5. PaaS Deployment
6. Serverless Deployment
7. Cloud-Native Architecture Principles

© Maximilien WALBECQ - Created with Marp e MarpX.

Scenario

Scenario

Web application in Python

Our application

```python
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/api")
def home():
    return jsonify({"message": "Hello from VM!"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```


© Maximilien WALBECQ - Created with Marp e MarpX.

Application Deployment on IaaS

Application Deployment on IaaS

Virtual Machine (again)

Approvisioning a VM is a IaaS solution because you have to:

- Defining the resources, scaling & load balancing
- Configure the infrastructure
- Manage the OS

Advantages:

- Full control
- Easy to deploy legacy application

Cons:

- Important set-up & maintenance

© Maximilien WALBECQ - Created with Marp e MarpX.

Demo time !

© Maximilien WALBECQ - Created with Marp e MarpX.

Application Deployment on IaaS

What if we need multiple applications ?

Let's say, you are a developer and you need to deploy several applications for your company. You can:

- Deploy on the same VM, better usage of your VM capacity but ...
  - Dependency conflict if you need the same package but with several different versions
  - Errors in one application can disrupt others.

- Create one VM per application, less dependancy conflict but ...
  - Lots of resources allocated
  - Multiplies the set-up and maintenance

© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Container-Based Deployment

What is a Container ?

Definition

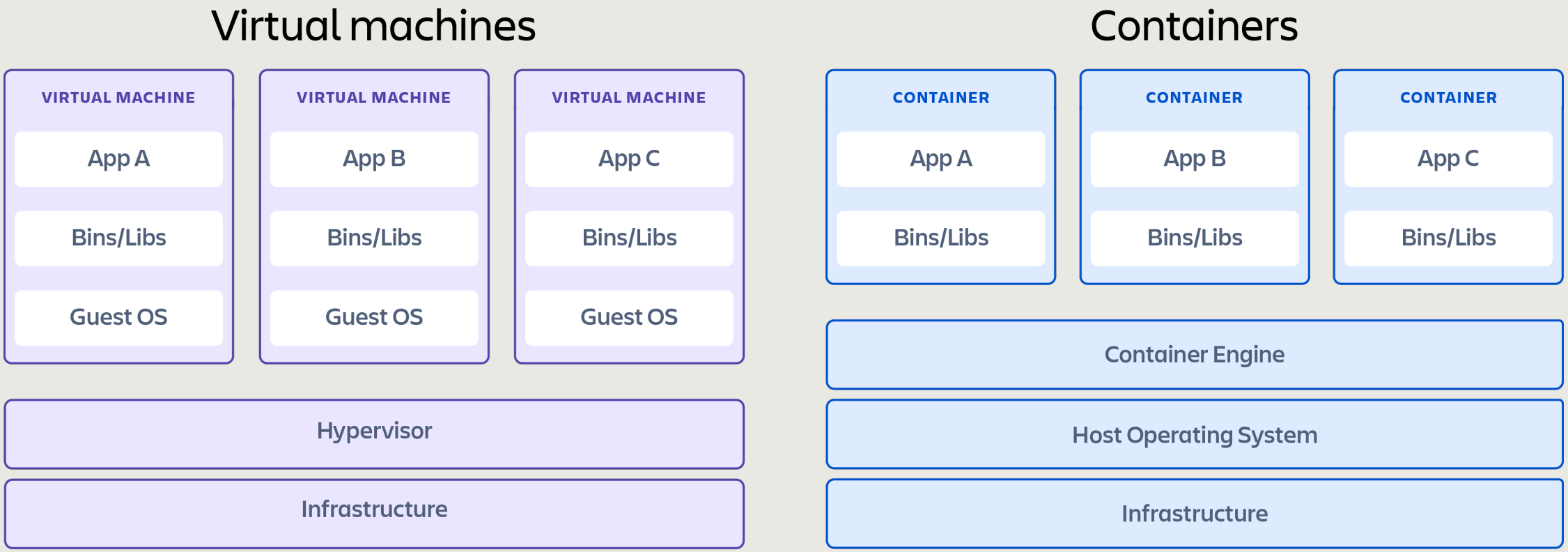
A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another

Advantages:

- Portability: Run consistently across different environments
- Lightweight: Share the host OS
  -> Fast startup: Containers launch in seconds
- Isolation: Each container runs its own application and dependencies without interfering with others

© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment



© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Required tools for Container

Image

Lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings

Image Registry

Centralized repository for storing and distributing container images.
It allows you to push (upload) and pull (download) images.

© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Required tools for Container

Container Runtime/Engine

The engine that runs containers, managing their lifecycle from creation to termination. Executes containerized applications in isolated environments.

Orchestration Platform

Automates deployment, scaling, load balancing, self-healing, and management of containerized applications across multiple hosts

We will not cover this one in this course.

© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Docker, the most widely used tool to build, run, and manage containers

- Image: Dockerfile
- Image Registry: Docker Hub, Azure Container Resgistry
- Container Runtime: Docker Engine
- Orchestration Platform: Kubernetes

© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Docker file

A Dockerfile is a text file containing instructions for building your source code.

```
FROM python:3.12                    # Defines a base for your image.
WORKDIR /app                        # Sets the working directory for instructions
COPY requirements.txt .             # Copies new files or directories
RUN pip install -r requirements.txt # Executes any commands in a new layer on top
                                    # of the current image
COPY . .
CMD ["python", "app.py"]            # Lets you define the default program that is
                                    # run once you start the container based on this image
```


© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Docker ignore

You can create a ".dockerignore" file
It lists files and directories that should be excluded when building the image (like a .gitignore).

```
__pycache__/
.env
.git/
node_modules/
```


© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Workflow

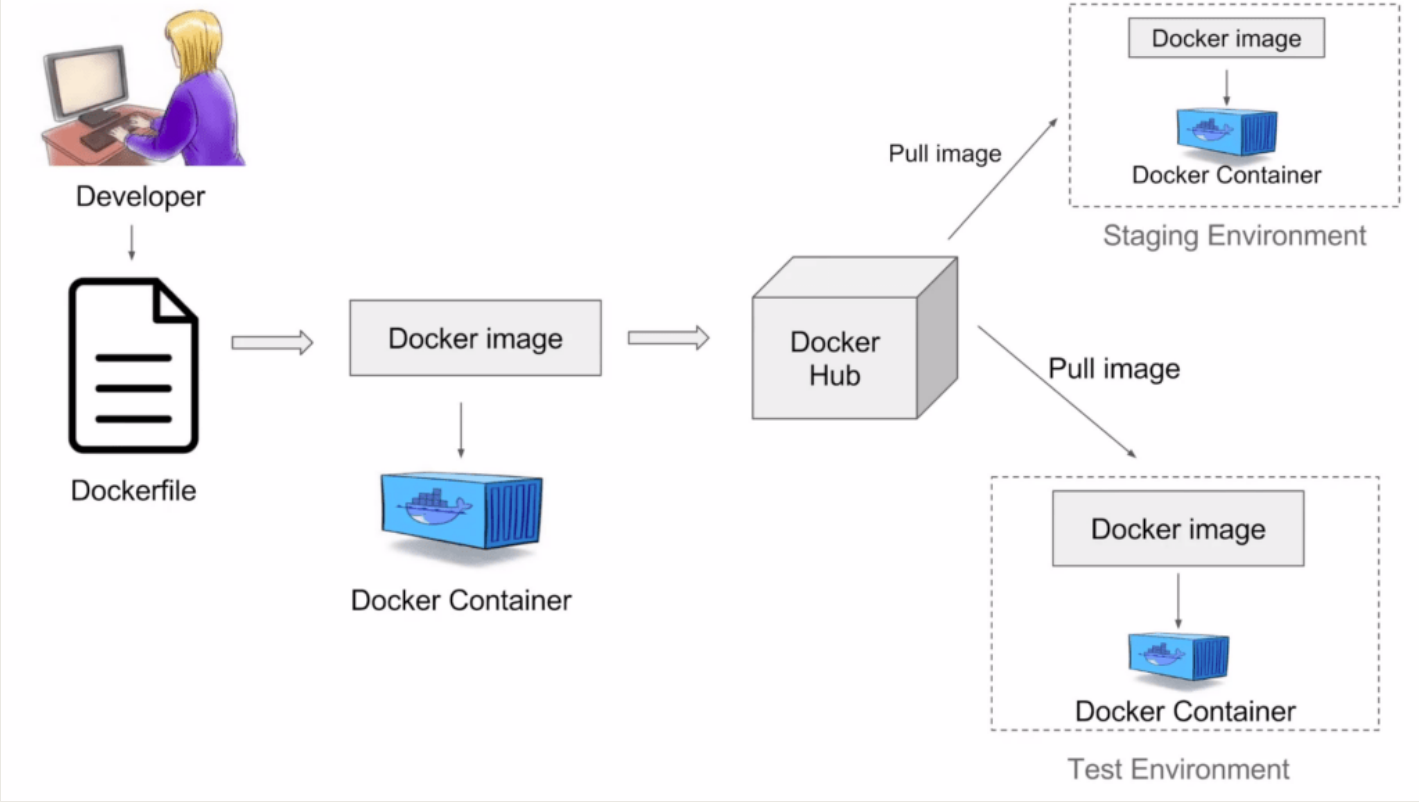


Fig. .1: Write -> Build -> Run -> Share -> Deploy

© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

Docker CLI

The command-line tool (docker) used to interact with the engine.

```
docker run                              # run a container
docker ps                               # list running container
docker stop                             # stop a container
docker rm                               # remove a container
docker build                            # build an image
docker tag                              # tag an image
docker push                             # push an image to the registry
```


© Maximilien WALBECQ - Created with Marp e MarpX.

Container-Based Deployment

The issue with container

Ok, we have a method for solving VM dependency issues.
However, that's a lot of configuration !

- We still use a VM
- We haven't even talked about orchestration. (load balancing, scaling, self-healing, ...)

What if we could give this **responsability** to the cloud ?

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Deployment

PaaS Deployment

Platform as a Service

Reminder from first course

- Environment to build and deploy applications
- Provider offers languages, frameworks, libraries, services
- No management of servers, network, or storage
- Control limited to the application and its runtime environment

© Maximilien WALBECQ - Created with Marp e MarpX.

Demo time !

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Deployment

The best approach: Container with PaaS

- Run **containerized applications** in a managed PaaS environment
- Platform handles:
  - Scheduling and orchestration of containers
  - Networking, service discovery, and load balancing between containers
  - Automatic scaling based on resource usage or traffic
- But you still have the flexibility & control of containers

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Deployment

Paas Container advantages

vs Traditional "code" App Service

- Full control over environment (Dockerfile)
- Any system dependency
- Any runtime/version
- Easily portable across clouds

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Deployment

Paas Container advantages

vs Self-managed Kubernetes

- No cluster management
- No complex network configuration
- No certificate management
- Auto-scaling without complex YAML configuration
- Built-in monitoring

© Maximilien WALBECQ - Created with Marp e MarpX.

Demo time !

© Maximilien WALBECQ - Created with Marp e MarpX.

Application on the Cloud

Overview

Application on the Cloud: Overview

Model	Advantages	Disadvantages	When to Choose
IaaS	- Full control over infrastructure - Works with almost any environment - High flexibility and customization	- Heavy maintenance - Scaling can be manual or complex - Higher human error risk	- When you need full control - For legacy app migration - For highly specific environments

© Maximilien WALBECQ - Created with Marp e MarpX.

Application on the Cloud: Overview

Model	Advantages	Disadvantages	When to Choose
PaaS	- Fast deployment, no server management - Built-in auto-scaling and load balancing - Native CI/CD and database integration - Runtime maintained by provider	- Less flexibility on OS/runtime configuration - Vendor lock-in - Not ideal for non-web workloads	- For web apps & APIs - For dev teams with little infra expertise - When time-to-market is key

© Maximilien WALBECQ - Created with Marp e MarpX.

Application on the Cloud: Overview

Model	Advantages	Disadvantages	When to Choose
PaaS Container	- Flexibility of containers + simplicity of PaaS - Full control over runtime environment - Scale to zero capabilities - Portable across cloud providers - No infrastructure management	- Requires Docker knowledge - Slightly more complex than traditional PaaS - May have cold start delays - Vendor-specific features can vary	- For microservice architectures - When you need custom dependencies - For modern cloud-native apps - Balance between control and simplicity

© Maximilien WALBECQ - Created with Marp e MarpX.

Serverless

Serverless Deployment

Reminder from last course

Definition

- Event-driven execution model
- No server management, auto-scaling
- Pay only per execution
- Focus on code
- Asynchronous, best for batch processing but not for application hosting

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Ecosystem: Managed Services

Other services you can use with your application

PaaS Ecosystem: Managed Services

Data

- **Databases**: Relational, NoSQL
- **Storage**: Blobs, files

Examples

- Azure SQL Database
- Azure Cosmos DB
- Azure Storage Account (yes it is also PaaS)

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Ecosystem: Managed Services

Integration

- **Message Queues**: Asynchronous processing, load leveling
- **Event Streams**: Real-time event-driven architectures
- **Workflows**: Orchestration and automation of services

Examples

- Azure Service Bus
- Azure Event Grid
- Azure Event Hubs
- Azure Logic Apps

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Ecosystem: Managed Services

Intelligence

- **Machine Learning Platforms**: Model training, deployment, and monitoring
- **AI APIs**: Prebuilt services for vision, speech, text, and predictions

Examples

- Azure Machine Learning
- Azure Cognitive Services
- Azure OpenAI Service

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Ecosystem: Managed Services

Identity & Secret

- **Identity Providers**: Authentication and single sign-on
- **Access Control**: Role-based or attribute-based authorization
- **Key & Secret Management**: Securely store credentials and certificates

Examples

- Microsoft Entra ID (Azure AD)
- Azure Key Vault
- Azure App Configuration

© Maximilien WALBECQ - Created with Marp e MarpX.

PaaS Ecosystem: Managed Services

Observability

- **Logging**: Centralized log collection and analysis
- **Metrics**: Real-time dashboards and performance indicators
- **Tracing**: Distributed request tracing across services
- **Alerting**: Automated notifications on issues or anomalies

Examples

- Azure Monitor
- Application Insights
- Log Analytics Workspace
- Azure Dashboard & Alerts

© Maximilien WALBECQ - Created with Marp e MarpX.

Cloud-Native Architecture Principles

What is cloud-native ?

Remember the characteristics of the Cloud ?
A cloud-native architecture is a way of designing, building, and running applications that fully leverage the cloud's dynamic, scalable, and distributed nature.

It's not a single technology (like containers or serverless), but rather a set of principles and patterns that make applications:

- Resilient (fault-tolerant and self-healing)
- Scalable (elastic and distributed)
- Observable (measurable and diagnosable)
- Automated (managed by code and policies, not humans)

This chapter is talking more about DevOps notion but since they are important for Cloud Computing, we will review them

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

It was created by "Heroku", a Platform as a Service cloud provider.
Source: https://12factor.net/

1. Codebase
2. Dependencies
3. Config
4. Backing Services
5. Build, Release, Run
6. Processes
7. Port Binding
8. Concurrency
9. Disposability
10. Dev/Prod Parity
11. Logs
12. Admin Processes

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

1. Codebase

One codebase tracked in revision control, many deploys

- A single codebase per app, stored in a version control system.
- Multiple deploys (production, staging, etc.) share the same codebase.
- Different apps should have different repositories.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

2. Dependencies

Explicitly declare and isolate dependencies

- Declare dependencies explicitly (e.g., in **requirements.txt**).
- Use dependency isolation (virtualenvs, containers, etc.).
- Never rely on implicit system packages.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

3. Config

Store config in the environment

- Configuration should not be hard-coded.
- Store secrets and environment-specific settings in environment variables.
- Enables portability without code changes.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

4. Backing Services

Treat backing services as attached resources

- Databases, queues, caches, and APIs are external resources.
- Access them via configuration.
- Easily swap services without changing code.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

5. Build, Release, Run

Strictly separate build and run stages

- Build: Convert code into an executable bundle.
- Release: Combine build with config for deployment.
- Run: Execute the app in the target environment.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

6. Processes

Execute the app as one or more stateless processes

- App should run as stateless, share-nothing processes.
- Persistent data goes to backing services.
- Scale easily by adding more process instances.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

7. Port Binding

Export services via port binding

- The app is self-contained and exposes HTTP (or another protocol) via a port.
- No need for a web server like Apache in front.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

8. Concurrency

Scale out via the process model

- Scale by running multiple process types and instances.
- Use process managers or container orchestration.
- Avoid multi-threaded designs when possible.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

9. Disposability

Maximize robustness with fast startup and graceful shutdown

- Processes should start fast and gracefully stop.
- Handle sudden termination.
- Enables rapid scaling and smooth deploys.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

10. Dev/Prod Parity

Keep development, staging, and production as similar as possible

- Same codebase, dependencies, and backing services.
- Automate deployment to reduce human error.
- Prevent “works on my machine” problems.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

11. Logs

Treat logs as event streams

- Apps should write logs to **stdout**.
- Let the environment handle log aggregation and storage.
- Focus on event streams, not log files.

© Maximilien WALBECQ - Created with Marp e MarpX.

The Twelve Factors App

12. Admin Processes

Run admin/management tasks as one-off processes

- Run migrations, scripts, or data imports as one-off tasks.
- Use the same codebase and environment.
- Keep processes disposable and stateless.

© Maximilien WALBECQ - Created with Marp e MarpX.

Thank you !

Let's take a break before the practice session!